Track A — Red Team Proposal

Project: Red-teaming a retrieval-augmented healthcare claims assistant **Track:** Track A — Red Team (attack and report on an AI application) **Proposed attack types:** Prompt injection and jailbreak, adversarial examples, data poisoning and backdoors, model and data extraction **Status:** Proposal for mentor review

---

1. Chosen Target

I am proposing that I build and then attack a single AI application: **a healthcare claims-triage and member-support assistant**. I want to call it *MedClaim Assist*.

The application is a realistic replica of a common production pattern. It is not a toy, and it is not a real vendor's product. It is a system I will build, host locally, and own, which means the engagement is fully authorised by construction: I am the owner of the target, so no third party is exposed and no rules of engagement need to be negotiated.

MedClaim Assist would consist of the following components, each of which represents a real architectural choice found in deployed AI products:

1. **A chat and agent layer.** An LLM-based conversational interface that answers member questions about claim status, benefit coverage, and required documentation. It is instructed by a system prompt that defines its scope, its refusal rules, and its escalation paths.
2. **A retrieval layer (RAG).** A document store of internal policy, benefit, and clinical-administration documents. The assistant retrieves relevant chunks and cites them in its answers. The knowledge base is versioned and periodically updated by human administrators.
3. **A document intake classifier.** A vision model that classifies uploaded claim attachments (referral letter, itemised invoice, referral form, clinical photograph) and routes them to the correct workflow. It also assigns an urgency flag.
4. **A claim risk-scoring classifier.** A text model that scores free-text claim descriptions for fraud and abuse risk, producing a score that gates whether a claim is auto-approved, sent to manual review, or blocked.
5. **A tool layer with real authority.** The agent can call three read-only tools: look up a member record, check a claim status, and fetch a document from a supplied URL. This is the part that gives the assistant genuine access rather than only generating text.
6. **A human-feedback retraining loop.** Operators rate assistant answers correct or incorrect. These ratings form a weekly retraining set for the models. This mirrors the feedback loops that real vendors advertise as "continuous improvement".
7. **Defensive controls already in place.** An input prompt-injection classifier, an output filter for sensitive patterns, and a NeMo-style rail that blocks a fixed list of topics. These are not token defences. I want to find out whether they hold.
8. **A synthetic datastore.** Fictional member records, fictional claims, fictional policy documents. I will seed a small number of **canary records** — obviously fake but highly distinctive strings and identifiers — so I can measure precisely whether and how protected data escapes.

The choice of healthcare claims is deliberate. It concentrates the highest-value failure modes in one system: personal and health data, financial decisions about real people, an urgent-care triage signal, and a retrieval corpus that an attacker can reach. It also lets me build on the Mediclinic threat assessment I completed in Week 5, so the threat model is grounded in work I have already done rather than invented for this project.

2. Why This Target

- **It is a full attack surface, not a single model.** A RAG assistant, a vision classifier, a text classifier, and a tool layer give me four genuinely different attack surfaces inside one application. I do not have to pretend that a chatbot is a complete AI system.
- **The components are real.** Every piece I have built or studied in the bootcamp maps onto one of these components: prompt injection guardrails, NeMo Guardrails, Garak probing, FGSM and PGD on CIFAR-10, BadNets and MNIST backdoors, data-poisoning pipelines, and a data-integrity pipeline.
- **Failures have asymmetric cost.** If the intake classifier misroutes a clinical photograph, a claim stalls. If the risk scorer is pushed below threshold, a fraudulent claim is paid. If the agent leaks a member record, that is a reportable privacy incident. A system where the mistakes are not all equal is a much better teaching artefact than a system where every error is just a wrong answer.
- **I own it.** Self-building removes the authorisation problem entirely, keeps all data synthetic, and makes every result reproducible by a mentor.

3. Objectives

The engagement has four objectives. They map one-to-one onto the four attack classes, and they are sequenced by likelihood and impact rather than by novelty.

Objective 1 — Break the LLM and agent layer. Determine whether an attacker can override the system prompt, make the assistant disclose its configuration, exfiltrate member data across an authorisation boundary, or abuse its tools. Then determine whether the deployed guardrails detect the attempt, and specifically whether they detect the *obfuscated* versions of it.

Objective 2 — Evade the models. Determine whether the document intake classifier and the claim risk classifier can be fooled with imperceptible or near-imperceptible perturbations, and whether an attack crafted on a surrogate model transfers to the deployed model.

Objective 3 — Corrupt the data lifecycle. Determine whether the retrieval corpus and the feedback loop can be poisoned so that the system produces attacker-chosen answers, and whether an implanted backdoor survives a retraining cycle.

Objective 4 — Steal the model or the data. Determine whether the model itself can be extracted through systematic querying, whether it memorises and leaks training content, and whether it can be used to infer whether a specific record was present in its training data.

4. Attack Classes and Planned Tests

I want to use all four attack classes, but not as an unstructured list. Each one attacks a different layer of the system, and together they cover the full path from input to decision to retraining.

Layer 1 — LLM and agent: prompt injection, jailbreak, excessive agency

ID	Test	What it probes
1.1	Direct instruction override ("ignore previous instructions")	Baseline instruction hierarchy
1.2	Indirect injection hidden inside a retrieved policy document	Whether the KB is treated as untrusted data

ID	Test	What it probes
1.3	System prompt and configuration extraction	Information disclosure of the control layer
1.4	Cross-member record access via the lookup tool	Authorisation enforcement and object-level access control
1.5	Tool parameter tampering and filter/retrieval injection	Whether tool inputs are validated
1.6	SSRF via the document-fetch tool	Whether the agent can be steered into fetching internal URLs
1.7	Guardrail evasion by encoding: base64, hex, reversed text, leetspeak, unicode homoglyphs	Whether the input classifier normalises before matching
1.8	Guardrail evasion by language: non-English and low-resource-language injections	Training-data language bias of the classifier
1.9	Roleplay and persona jailbreaks (DAN-style, "developer mode", fictional framing)	Whether refusal is enforced or merely stylised
1.10	Instruction smuggling across turns and across retrieved documents	Multi-turn and multi-hop instruction persistence
1.11	Goal hijacking: silently redefining the assistant's task	Task integrity, not just refusal behaviour
1.12	Exfiltration to an attacker-controlled sink I control	Whether output can be steered to leak data
1.13	Untrusted content passthrough and unsafe rendering	Whether model output is treated as untrusted at the sink
1.14	Canary recovery: can the seeded canary member data be extracted at all	The single most important question in this layer

For every test in this layer I will record not just whether the attack succeeded, but whether the guardrail blocked it, whether it blocked it *for the right reason*, and how confident the guardrail was. My Week 7 experiment showed a detector that was 100% confident while passing a working exfiltration prompt, so confidence is not evidence and I will report the two separately.

Layer 2 — Adversarial examples against the classifiers

ID	Test	What it probes
2.1	FGSM and PGD on the document intake classifier	Single-step and iterative perturbation of a vision model
2.2	Targeted attack to force a specific wrong class	Controllability, not just misclassification
2.3	Adversarial text perturbations on the claim risk scorer	Whether text models are perturbable at all, or only via encodings
2.4	Homoglyph, zero-width, and typo substitution on the risk scorer	Evasion that is invisible to a human reviewer
2.5	Threshold-targeted attack: push a fraudulent claim below the review gate	Whether the score can be steered to a business-relevant boundary

ID	Test	What it probes
2.6	Transferability: craft on a surrogate, test on the deployed model	Whether the attack generalises beyond one model
2.7	Physical-world note: what survives a screenshot, a resave, and a re-scan	Whether the attack is a lab artefact or a real-world one

I will measure the perturbation budget, the misclassification rate, and the transfer success rate, and I will record the robustness numbers honestly, including the fact that PGD-based robustness is an upper bound on the true robustness, as I noted after the FGSM lab in Week 7.

Layer 3 — Data poisoning and backdoors

ID	Test	What it probes
3.1	Poison the retrieval corpus with a crafted policy document	Integrity of the knowledge base and its review process
3.2	Poison the corpus to make a specific false coverage answer appear authoritative	Whether citations can be made to lie convincingly
3.3	Abuse the feedback loop to corrupt the retraining set	Whether "human feedback" is actually human
3.4	Clean-label and label-flipping poisoning of the claims dataset	Integrity of the training data supply chain
3.5	Backdoor implantation (BadNets-style trigger) in the risk scorer	Whether a covert trigger can be installed and hidden
3.6	Retrain after backdoor implantation and re-test	**Persistence.** A backdoor that survives one retraining cycle is a much more serious finding than one that does not
3.7	Integrity-control bypass: does the hashing pipeline from Week 8 catch this	Whether my own Week 8 data-integrity control would have detected the poisoning

The persistence test at 3.6 is the one I care about most in this layer. The real question for a production system is not "can a backdoor be planted" but "does it still work next quarter".

Layer 4 — Model and data extraction, and privacy inference

ID	Test	What it probes
4.1	Model extraction through systematic querying	Whether the model can be cloned well enough to substitute for the real one
4.2	Verbatim memorisation probing for training-set content	Whether sensitive text is recoverable from the weights
4.3	Membership inference: was a specific synthetic member record in training?	The real healthcare question, independent of any attack
4.4	Vector-store and embedding leakage	Whether the retrieval index itself discloses data
4.5	Cross-tenant retrieval isolation	Whether one request can reach another member's documents

Membership inference at 4.3 is the test I expect to matter most in a real deployment, and it is also the test a healthcare client would care about most. It does not require breaking anything.

5. Rules of Engagement and Ethics

Because I am building the target, authorisation is straightforward, but I will still hold the engagement to explicit limits:

- **Authorised target only.** Only MedClaim Assist is in scope. No real vendor product, no public shared instance, and no third-party endpoint.
- **Synthetic data only.** No real patient data, no real claims, no real credentials, and no scraping of real records. All PII in the datastore is fictional. Canary records are obviously fictional by construction.
- **Isolated deployment.** Local or private-hosting only. I will not expose the application to the public internet, which removes the risk of my testing being mistaken for a real service or being reached by someone else.
- **No denial of service.** I will not run resource-exhaustion tests, will cap request volume, and will set a spending ceiling on any paid model API. Availability findings will be argued from design and code review rather than from taking the system down.
- **Bounded payload exfiltration.** Where a test involves data leaving the system, the sink is a server I control, holding only synthetic data.
- **No naming real vendors in public.** Findings will be reported to my mentors first. Nothing goes into a public write-up until it has been reviewed, and no finding will be published in a way that would help someone attack an unnamed real product.
- **Defence-oriented framing.** Every finding ships with a remediation. A red-team report that lists problems without proposing fixes is of limited use, and this project is about improving the system, not about publishing a score.

6. Methodology and Tooling

The work follows a red-team loop rather than a linear list of tests: **enumerate the surface, hypothesise a weakness, execute an authorised test, record the evidence, map the result to ATLAS, propose a fix, then re-test the fix.**

- **Surface enumeration** from the application's own code and configuration, so the threat model describes the system I actually built.
- **Hypothesis-driven testing.** Each test states in advance what result would falsify it, which stops me from reporting noise as a finding.
- **Reproducible artefacts.** Every test is a notebook or script with fixed seeds and saved outputs, so a mentor can re-run any result. This is the same standard I applied to the FGSM and MNIST labs.
- **Structured attack library.** Each entry records: identifier, layer, technique, MITRE ATLAS mapping, preconditions, payload, expected result, actual result, success or fail, severity, and evidence path.
- **Severity rating** using a consistent scale, considering both technical impact and realistic exploitability, with the business consequence stated in plain language.
- **Retest after remediation**, reported separately from the original finding, so the report shows verified improvement rather than promised improvement.

Tooling will reuse what the bootcamp already covered rather than adding new dependencies: the prompt-injection detector and Garak for automated probing, NeMo Guardrails configuration review, FGSM/PGD implementations,

and the SHA-256 data-integrity pipeline for the poisoning tests.

7. Deliverables

1. **Threat model and attack-surface map.** Data-flow diagram of the application, trust boundaries, entry points, and the data each component can reach. This is the document that justifies every subsequent test.
2. **Reproducible attack harness.** One notebook or script per attack class, with seeds fixed and outputs saved.
3. **Attack library.** The structured record of every test and its result, machine-readable so it can be re-run or extended.
4. **Findings report.** Executive summary for a non-technical reader, followed by technical findings with reproduction steps, evidence, severity, and ATLAS mapping. This is the primary Track A deliverable.
5. **Prioritised remediation plan with retest results.** Ordered by risk reduction per unit of engineering effort, with evidence that each fix works.
6. **Detection and monitoring pack.** Suggested log fields, canary-token strategy, and alert rules so the organisation would know it was under attack. Detection is the control that turns a one-off fix into a durable capability.
7. **Reflection.** What surprised me, what I got wrong, and what the exercise changed about how I think about securing AI systems.

8. Timeline

Week 1 — Build and baseline. Threat model first, then the application. Implement the LLM and agent layer, the RAG pipeline, the two classifiers, the tools, the guardrails, and the synthetic datastore with canaries. Record baseline behaviour and baseline accuracy for both classifiers. No attacks yet; a baseline is what makes every later number meaningful.

Week 2 — Layer 1 attacks. Execute the prompt injection, jailbreak, exfiltration, and tool-abuse suite. Measure guardrail block rate, bypass rate, and false-positive rate separately. Attempt canary recovery.

Week 3 — Layers 2 and 3. Run the adversarial-example campaign against both classifiers, including transferability and the threshold-targeted test. Then run the poisoning and backdoor campaign, including the persistence retest.

Week 4 — Layer 4, reporting, and retest. Model extraction, memorisation, and membership inference. Then the report, the remediation plan, the monitoring pack, and the retest of every fix I can verify.

9. Success Measures

The project succeeds if it produces trustworthy evidence, not if it produces a long list of failures. Concretely:

- **Guardrail bypass rate per attack family**, reported with numerator and denominator, plus the false-positive rate on benign prompts, because a control that blocks everything has not solved anything.
- **Attack success rate per layer**, with each success backed by captured evidence.
- **Perturbation budget and misclassification rate** for each adversarial test, and the transfer success rate from surrogate to deployed model.
- **Whether the backdoor survives retraining.** A binary, unambiguous, and highly informative result either way.
- **Canary recovery result.** Whether the seeded canary data can be extracted, and by which technique. This is the finding I would most want a mentor to read.

- **Membership inference advantage** for the record-membership question, compared against a random-guessing baseline.
- **Time to detect.** Whether the monitoring pack would actually have flagged the successful attacks. A successful attack that was invisible to logging is the most important finding in the whole report.
- **Verified fixes.** Every remediation retested, with before-and-after numbers.

10. Risks and Limitations

I want to be honest about what this project cannot show.

- **A self-built target is not a production system.** Its defences will be the defences I thought to build. Finding a weakness in my own weak control proves little about a well-defended product. I will state clearly which findings are architectural, meaning they would apply to any implementation of this pattern, and which are artefacts of my simplified build.
- **A small model will behave differently from a frontier model.** Robustness numbers against a small classifier do not transfer automatically to a large one. I will keep the model sizes and configurations documented so the results are interpretable and not over-generalised.
- **I am one attacker with limited time.** The attack library will be a sample, not an exhaustive assessment. It is a foundation for a real programme, not a substitute for one.
- **Defence is not fully in scope.** Track A is attack and report. I will propose remediations and verify the ones that are cheap to verify, but hardening the system properly is Track B's job, and I would rather hand Track B a precise work list than pretend the red-team pass fixed anything.
- **Synthetic data weakens the privacy findings.** Exfiltrating a fictional member record is a proof of mechanism, not a breach. The membership-inference and memorisation results transfer conceptually; the drama does not.

11. What I Expect to Find

I expect the guardrails to catch the obvious attacks and fail on the awkward ones, exactly as my Week 7 detector experiment predicted. I expect the indirect injection through the retrieval corpus to be the most successful Layer 1 technique, because the system prompt and the retrieved documents are concatenated into one context and the model has no principled way to distinguish instructions from data. I expect a working evasion of the claim risk threshold, because that boundary is exactly where a gradient attack is most effective and exactly where the business cares. I expect the backdoor to survive one retraining cycle, because that is what the literature and my own Week 8 results suggest. And I expect the canary data to be extractable, which would mean the guardrails are providing the appearance of protection rather than the substance.

If those predictions turn out to be wrong, that is a genuinely interesting result and I will report it as one. The point of the hypothesis is to be falsified, not to be confirmed.

12. Personal Reflection

Track A is the track that matches how I have trained for this, but not for the reason I expected when I chose it. I did not pick it because red teaming sounds dramatic. I picked it because the last twelve weeks repeatedly showed me the same pattern from different angles. In Week 3 a classifier's accuracy changed with the test split. In Week 7 a guardrail passed a working exfiltration prompt at 100% confidence. In Week 8 a small poisoned dataset implanted a trigger that quietly changed the model's behaviour. In Week 10 a photo with no metadata told a different story

from its pixels. Every one of those was a finding that a conventional review would have missed, because the system did not look broken. It looked like it was working, and it returned confident answers while it was wrong.

That is the thing I want to investigate properly. The gap between how an AI system appears and how it behaves is where the security risk lives, and a red-team engagement is the only way to measure that gap honestly. Attacking a system I built also forces a discipline I have not had to practise yet: I have to make the target realistic enough that the findings mean something, which means I have to think like a defender before I can think like an attacker.

I also want to practise the part of red teaming that is not technical. A finding that cannot be explained to the person who has to act on it does not get fixed. So I want every result expressed twice: once with the technical evidence for the engineer who will verify it, and once in plain language with a concrete business consequence for the person who will fund the fix. In Week 5 I assessed Mediclinic's threat landscape; in Week 6 and Week 7 I analysed the Old Mutual red-team disclosure and wrote about what a real red team should find. This project is the applied version of those reflections, and it is where I expect to learn the most.

13. Conclusion

I propose to build and red-team MedClaim Assist, a retrieval-augmented healthcare claims assistant, across all four attack classes: prompt injection and jailbreak, adversarial examples, data poisoning and backdoors, and model and data extraction. The target is self-built, self-hosted, and seeded only with synthetic data, so the engagement is fully authorised, safely bounded, and reproducible.

The value of this project is not the number of vulnerabilities I find. It is the demonstration that an AI application can fail in four different ways at four different layers, that a confident model can be wrong and a strong guardrail can be a pattern-matcher, and that the practical security question is not whether an attack works but whether the organisation would notice, how fast, and how little it would cost. If I can answer those three questions with reproducible evidence and a prioritised fix list, the project has done its job.

Approval requested for: the target selection and architecture in Section 1, the four attack classes in Section 4, the rules of engagement in Section 5, and the four-week timeline in Section 8. I am happy to narrow the scope to a single layer if a broader engagement would not fit the schedule.
